

FACULDADE IMACULADA CATÓLICA DO RECIFE - FICR

**CURSO DE TECNOLOGIA EM ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
PROGRAMA EMBARQUE DIGITAL — 1º PERÍODO**

**DOCUMENTAÇÃO/RELATÓRIO TÉCNICO DE DESENVOLVIMENTO DO
PROJETO: SISTEMA DE GESTÃO PARA MERCADINHO.
DESENVOLVIMENTO DE SOFTWARE EM LINGUAGEM C**

Relatório Técnico
apresentado como requisito
de avaliação para a disciplina
de Programação. Ministrada
pelo Professor Jonatha.

RECIFE - PE 2026

Trabalho elaborado pelos estudantes:

Carlos Eduardo Alves Nunes
Thays Fernanda Martins Candido da Silva
Ana Vitória Ramos de Mendonça
Gabriel Santos Gonçalves
Charlys Robert de Melo Gueiros
Miguel Rodrigues de Oliveira

SUMÁRIO

1.INTRODUÇÃO.....	3
2.ESPECIFICAÇÃO TÉCNICA (O QUE FOI USADO).....	4
2.1Modelagem Dinâmica de Dados (Structs).....	4
3.ESTRUTURA DO PROJETO.....	5
3.1 O que foi salvo?.....	5
3.2 Onde foi salvo?.....	5
3.3 tabela.....	6
4.COMO O SISTEMA FUNCIONA (MÓDULOS).....	7
4.1 Módulo de Clientes.....	7
4.2 Módulo de Produtos e Estoque.....	7
4.3 Módulo de Pedidos e Vendas.....	7
5.FLUXO DE DADOS E SEGURANÇA DO CÓDIGO.....	8
6.Cuidados com Erros e Segurança.....	8
7.CICLO DE EXECUÇÃO DO SOFTWARE (COMO VAI RODAR).....	9
8.Dimensionamento de Memória.....	10
9.Controle de Buffer e Estabilidade do Terminal.....	10
10.MANUAL DE OPERAÇÃO.....	11
10.1 Fluxo de Cadastro e Entrada de Dados.....	11
10.2 Movimentação de Estoque e Auditoria.....	11
11.CONCLUSÃO.....	12
12.Referências.....	12
13.Git hub do projeto.....	12

1. INTRODUÇÃO

Este documento explica como foi desenvolvido o Sistema de Gestão para Mercadinho. O software foi feito na linguagem C e serve para ajudar pequenos comércios a controlarem suas tarefas do dia a dia, como cadastro de clientes, estoque de produtos e fluxo de vendas.

O objetivo do projeto é aplicar os conceitos de programação estruturada e também o uso de ferramentas de controle de versão (Git e GitHub), que são fundamentais para a rotina de um desenvolvedor.

O projeto foi criado para aplicar na prática o que aprendemos em sala de aula, como a divisão do código em partes (módulos), o uso de structs, vetores e a gravação de dados em arquivos para que as informações não se percam quando o programa for fechado. O sistema resolve três problemas principais do mercadinho:

- * Falta de controle do que tem no estoque.
- * Falta de um cadastro organizado de clientes.
- * Dificuldade para somar os valores e fechar uma venda.

2. ESPECIFICAÇÃO TÉCNICA (O QUE FOI USADO)

Bibliotecas Padrão da Linguagem C Para garantir a portabilidade e a execução eficiente dos algoritmos, foram mobilizadas apenas bibliotecas nativas do compilador C, mapeadas conforme suas responsabilidades específicas no projeto:

<stdio.h>: Responsável pelo controle de fluxos de entrada e saída padrão (comandos printf e scanf), além de gerenciar os ponteiros de arquivos para leitura e escrita em disco rígido (fopen, fclose, fread, fwrite, fprintf).

<string.h>: Utilizada no processamento, cópia e comparação de arrays de caracteres, aplicando funções como strcpy/strncpy para atribuição de dados textuais e strcmp/strstr para motores de busca e filtros de pesquisa.

<time.h>: Mobilizada para capturar as marcas temporais da CPU através da estrutura struct tm. Permite o cálculo regressivo de datas (controle de validade de perecíveis em dias) e o registro automático de data e hora em tempo real nos logs de auditoria.

<ctype.h>: Aplicada no tratamento de caracteres individuais através de funções de normalização como tolower e toupper, mitigando falhas na busca de dados por incompatibilidade de caixa alta ou baixa.

<locale.h>: Garante a conformidade estética da interface gráfica textual no terminal através da instrução setlocale(LC_ALL, "portuguese"), habilitando o suporte nativo a acentuações gráficas e caracteres especiais da língua portuguesa.

2.1 Modelagem Dinâmica de Dados (Structs):

O agrupamento dos dados na memória RAM baseia-se no princípio de estruturas heterogêneas (typedef struct). Foram modeladas três entidades diferentes para juntar as regras de negócio operacionais:

Alimento: Estrutura que unifica metadados comerciais (código, nome, categoria, peso, quantidade, preço) a metadados cronológicos internos calculados em tempo de execução (dia, mês, ano de validade, dias restantes para o vencimento e indicador numérico de prioridade de consumo).

Movimentação: Entidade responsável por atuar como um guia de acompanhamento, armazenando o identificador do produto, a cadeia de caracteres da ação realizada (CADASTRO, ENTRADA, SAÍDA, EXCLUSÃO),

o quantidade movimentada e o registro cronológico exato da operação no formato DD/MM/AAAA.

Cliente: Estrutura administrativa complexa dotada de 20 campos distintos que se ramificam entre dados de identificação civil (Nome, CPF, RG, Nascimento), informações de contato e geolocalização completa (Endereço, Bairro, CEP, Cidade, UF), parâmetros financeiros de risco (Limite de Crédito) e sinalizadores de controle lógico (ativo e tipo de pessoa).

O QUE FOI SALVO E ONDE ONDE FOI SALVO (ESTRUTURA DO PROJETO)

Para o código não ficar gigante e difícil de entender, o sistema foi dividido em partes menores para ficar organizado. Cada módulo cuida de uma parte do mercadinho. A seguir, está a explicação do que foi salvo e como esses arquivos estão guardados.

O que foi salvo?

Arquivos de Código (.h e .c): Onde está toda a lógica do programa (menus, cadastros e contas). Os arquivos *.h* guardam os modelos (structs) e os nomes das funções, e os arquivos *.c* guardam o código funcionando.

Arquivos de Dados (.dat): Arquivos binários criados pelo próprio programa na pasta *data/*. Eles servem para salvar as informações de clientes, produtos e vendas. Assim, mesmo se o computador desligar, os dados não somem.

Arquivo de Instruções (README.md): Um texto explicativo que fica na página inicial do projeto no *GitHub* para ensinar outras pessoas a usarem o sistema

Onde foi salvo? (Uso do *Git* e *GitHub*):

Os arquivos foram salvos de duas formas:

Localmente: Na máquina onde o sistema foi programado, seguindo uma estrutura de pastas organizada.

Na Nuvem (*GitHub*): Foi usado o *Git* para controlar as versões do código (atualizar e corrigir erros) e tudo foi enviado para um repositório online no *GitHub*. Isso garante a segurança do código e permite fácil acesso aos arquivos direto da nuvem.

README. md	main.c	clientes.c	produtos. c	pedidos.c	data/
Instruções iniciais do projeto	Menu principal que liga todo o sistema	Como o cadastro de clientes funciona	Como o controle de estoque funciona	Como as vendas funcionam	Pasta onde os dados ficam salvos
		clientes.h Modelos e funções de clientes	produtos.h Modelos e funções de produtos	pedidos.h Modelos e funções de vendas	clientes.dat Arquivo com os clientes salvos
				pedidos.dat Arquivo com o histórico de vendas	produtos.dat Arquivo com os produtos salvos

COMO O SISTEMA FUNCIONA (MÓDULOS)

O programa roda em uma tela de texto (console) e possui três áreas principais que o usuário acessa digitando números no menu:

Módulo de Clientes

Cuida das pessoas que compram no mercadinho.

O que faz: Cadastra novos clientes, procura por cliente já cadastrado, atualiza os dados ou remove o cliente do sistema.

Dados salvos: ID (número do cliente), Nome, CPF, Telefone e E-mail.

Módulo de Produtos e Estoque

Controla as mercadorias do mercadinho.

O que faz: Cadastra produtos, muda preços, mostra o que tem nas prateleiras e diminui a quantidade guardada de forma automática quando uma venda acontece.

Dados salvos: Código de Barras (SKU), Nome do Produto, Preço de Venda, Quantidade no Estoque e Estoque Mínimo (para avisar quando o produto estiver acabando).

Módulo de Pedidos e Vendas

É o caixa do mercadinho.

O que faz: Abre uma nova venda, vincula o cliente que está comprando, adiciona os produtos e a quantidade de cada um, soma o valor total e finaliza a compra atualizando o estoque.

FLUXO DE DADOS E SEGURANÇA DO CÓDIGO

a tabela abaixo mostra como os dados entram, como são processados e o que o sistema exibe:

Módulo / Ação	Cadastrar Produto	Vender Item	Fechar Pedido.
O que o usuário digita (Entrada)	Código, Nome, Preço e Quantidade.	Código do Produto e Quantidade vendida.	ID do Cliente e confirmação.
O que o sistema faz (Processo)	Verifica se o código já existe; checa se o preço é maior que zero; salva no arquivo produtos.dat.	Procura o produto; vê se tem quantidade suficiente no estoque; diminui o estoque atual.	Junta todos os itens; calcula o valor total da compra; salva a venda em pedidos.dat.
O que aparece na tela (Saída)	Mensagem de "Produto cadastrado com sucesso"	Estoque atualizado no arquivo e item adicionado ao carrinho.	Mostra o cupom fiscal com a lista de itens e o total a pagar.

Cuidados com Erros e Segurança

Foi adicionado travas no código em C para evitar que o programa feche sozinho ou trave:

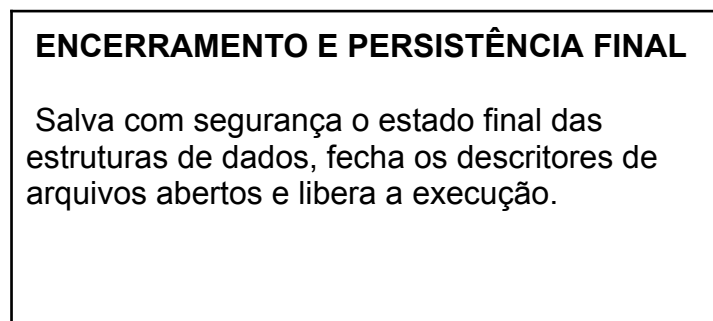
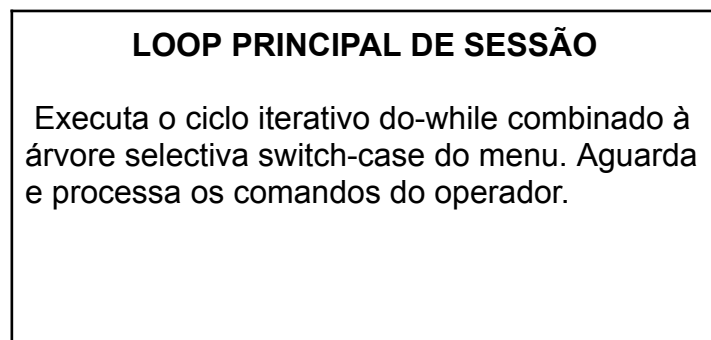
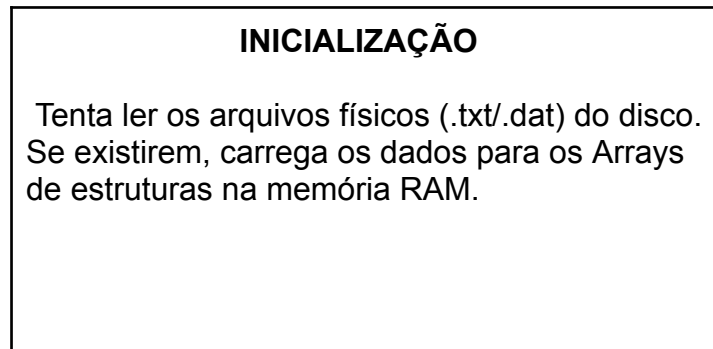
Uso do fgets: Evitando usar a função gets (que pode quebrar a memória do programa). Foi utilizado fgets para limitar o tamanho do texto que o usuário digita.

Proteção contra Estoque Negativo: O sistema não deixa fazer uma venda se o cliente pedir mais itens do que o mercadinho tem no estoque.

Verificação de Arquivos: Sempre que o programa tenta abrir os arquivos .dat, ele checa se o arquivo realmente abriu. Se houver erro (como falta de permissão na pasta), o programa avisa o usuário em vez de fechar com erro.

CICLO DE EXECUÇÃO DO SOFTWARE (COMO VAI RODAR)

Inicialização e Gerenciamento de Memória A execução do programa é regida por uma arquitetura de ciclo contínuo estruturada em três fases bem delimitadas dentro da função executora do sistema:



Dimensionamento de Memória

O tamanho da memória é definido de forma estática por meio de diretivas de pré-compilação, estabelecendo um limite fixo seguro (`#define MAX 100`). Essa configuração evita estouros de pilha (*stack overflow*) e garante o comportamento previsível do sistema durante a execução no terminal.

Controle de Buffer e Estabilidade do Terminal

Um dos maiores desafios em aplicações interativas de console em C é a instabilidade gerada por caracteres residuais no buffer de entrada (stdin). Para blindar o sistema contra leituras fantasmas de capturas textuais a leituras numéricas, os códigos implementam rotinas dedicadas de limpeza:

```
void limpar_buffer(void)
{
    int c;
    while ((c = getchar()) != '\n' && c != EOF);
}
```

Esta instrução limpa os resíduos deixados pela tecla Enter, garantindo que as funções de leitura de strings baseadas em fgets funcionem corretamente sem pular campos na interface de digitação do usuário.

MANUAL DE OPERAÇÃO

Fluxo de Cadastro e Entrada de Dados

A interface do terminal opera de maneira sequencial. Ao selecionar a opção de cadastro no menu principal, o sistema gera de forma automática o identificador (ID) sequencial e incremental do registro. Para o cadastro de clientes, a tela exibe campos específicos de acordo com a seleção de Pessoa Física ou Jurídica, alternando a coleta de dados entre CPF e CNPJ. A efetivação do registro ocorre apenas após a confirmação manual com o comando **[S/N]**.

Movimentação de Estoque e Auditoria

O registro de entrada ou saída de mercadorias exige a inserção do ID do item. O sistema realiza a busca e exibe o saldo atual em tela. Após o preenchimento da quantidade desejada, o software avalia a viabilidade da operação: caso uma saída resulte em saldo negativo, a execução é bloqueada imediatamente com um alerta visual. As operações válidas atualizam o saldo em tempo real e geram um registro automático de auditoria com data e hora no histórico do sistema.

CONCLUSÃO

O desenvolvimento deste projeto foi muito importante para entender como criar um software organizado e funcional usando a linguagem C. A separação do código em módulos facilitou encontrar erros e trabalhar no sistema por etapas. As validações de segurança e o salvamento de dados em arquivos garantiram que o programa funcione como um sistema real de comércio, cumprindo os objetivos propostos na disciplina.

Referências

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. NBR 14724: Informação e documentação - Trabalhos acadêmicos - Apresentação. Rio de Janeiro: ABNT, 2011.

KERNIGHAN, Brian W.; RITCHIE, Dennis M. C: A Linguagem de Programação Padrão ANSI. 2. ed. Rio de Janeiro: Campus, 1990.

SCHILDT, Herbert. C COMPLETO E TOTAL. 3. ed. São Paulo: Makron Books, 1997.

Git hub do projeto

<https://github.com/eduualvesss/Projeto-Mercadinho>